Amiyo-Go

Marketplace Thesis Documentation

Architecture, features, workflows, implementation status, testing, deployment, and future scope

Prepared for project documentation and production-readiness review

Generated from AMIYO_GO_THESIS_DOCUMENTATION.md

Date: 2026-06-04

Amiyo-Go Marketplace Thesis Documentation

Project: Amiyo-Go Multi-Vendor Marketplace

Document type: Thesis-style technical documentation

Last updated: 2026-06-04

Prepared for: Product, engineering, operations, vendor, and admin review

Abstract

Amiyo-Go is a role-based multi-vendor marketplace system designed to support customer shopping, vendor seller-center operations, and admin marketplace governance. The project has evolved beyond a simple ecommerce catalog into a marketplace operating system with modules for catalog management, vendor onboarding, order fulfillment, courier workflow, returns, payments, promotions, trust and safety, analytics, staff permissions, and documentation/training.

The application follows a modular monolith architecture. The frontend is built with React, React Router, Tailwind CSS, reusable UI components, and role-specific layouts. The backend is a Node.js and Express API layer using MongoDB through Mongoose and native MongoDB access. Selected background workflows use BullMQ/Redis where configured. The system includes adapter-ready integrations for courier providers, payment providers, notification channels, object storage, and search provider abstraction.

This documentation records the full implementation scope, current architecture, feature inventory, role workflows, data model map, API surface, quality practices, deployment notes, known limitations, and future production-hardening roadmap. It is intended to work as a project thesis, handover document, and production-readiness reference.

Keywords

Marketplace, ecommerce, multi-vendor, seller center, admin dashboard, logistics, COD, returns, trust and safety, analytics, promotions, React, Node.js, Express, MongoDB, Firebase, BullMQ.

Table of Contents

1. Introduction
2. Project Goals and Scope
3. Stakeholder and Role Analysis
4. Technology Stack
5. System Architecture and UI Screenshots
6. Frontend Architecture

7. Backend Architecture
8. Database and Persistence Model
9. Feature Documentation by Role
10. Business Workflows
11. API and Service Map
12. Security, Trust, and Permissions
13. Logistics and Courier Integration
14. Promotions, Growth, and Personalization
15. Analytics and Reporting
16. Testing and Quality Assurance
17. Deployment and Environment Configuration
18. Documentation and Training Layer
19. Production-Readiness Assessment
20. Limitations and Future Work
21. Conclusion
22. Appendices

Chapter 1: Introduction

1.1 Background

Modern marketplaces are not only product listing websites. They are operational platforms where customers, sellers, and platform operators coordinate through structured workflows. A Daraz-level marketplace needs customer discovery and checkout, seller-center controls, logistics state tracking, finance settlement, dispute handling, campaign tools, analytics, and staff governance.

Amiyo-Go is designed around that concept. The project connects customer shopping actions, vendor operating tools, and admin governance into one system. The core value is not only that products can be purchased, but that the marketplace can be managed after the order is placed.

1.2 Problem Statement

A normal ecommerce application usually solves only these problems:

- Display products.
- Add products to cart.
- Place an order.
- Let an admin view orders.

A real marketplace must solve many more:

- Sellers need onboarding, KYC, product moderation, stock management, packing, finance, promotions, and reputation tools.
- Customers need trustworthy shopping, order tracking, invoices, returns, refunds, support, reviews, saved addresses, alerts, loyalty, and notifications.
- Admins need vendor approval, moderation queues, dispatch controls, COD reconciliation, payment verification, payout management, trust and safety, analytics, staff permissions, and audit logs.
- Operators need clear documentation and training so each role understands what they should see and do.

Amiyo-Go addresses these needs through a modular marketplace workflow.

1.3 Project Aim

The aim of Amiyo-Go is to provide a production-oriented marketplace foundation that can operate with manual workflows first and gradually attach paid/external integrations such as courier APIs, payment gateways, SMS providers, object storage, and search engines.

1.4 Thesis Objective

This document has four objectives:

1. Explain the complete project architecture.

2. Document all major customer, vendor, and admin features.
3. Record operational workflows and implementation status.
4. Provide a production-readiness reference for future hardening.

Chapter 2: Project Goals and Scope

2.1 Functional Goals

The main functional goals are:

- Build a three-role marketplace: customer, vendor, and admin.
- Allow customers to browse, search, purchase, track, return, review, and get support.
- Allow vendors to manage shop, KYC, catalog, inventory, orders, fulfillment, returns, finance, marketing, Q&A, reviews, and reports.
- Allow admins to manage vendors, products, orders, returns, payouts, COD, logistics, promotions, trust, staff, analytics, and platform settings.
- Support customer-facing shop storefronts with follow/unfollow and vendor products.
- Support marketplace growth through promotions, flash sales, loyalty, recommendations, alerts, and notifications.
- Support logistics through internal shipment state machines and adapter-ready courier booking.
- Support role learning through Amiyo-Go University with protected vendor/admin learning content.

2.2 Non-Functional Goals

Important non-functional goals include:

- Maintainable modular monolith design.
- Role-based access control.
- Secure API boundaries.
- Responsive frontend UI.
- Progressive Web App readiness.
- Auditable admin and finance actions.
- Extensible integration adapters.
- Documentation-first handover.
- Testable frontend and backend modules.

2.3 Scope Boundary

The project currently includes many production-grade foundations, but some external systems remain environment-dependent:

- MongoDB is the current database. PostgreSQL is not the current implementation.
- Redis and BullMQ exist for selected background workflows, but they are not yet universal for every event.
- Courier APIs are adapter-ready. Manual and internal courier workflows remain valid without paid courier API activation.
- Payment provider adapters exist, but production payment depth depends on provider credentials and webhook configuration.
- Email, push, and optional SMS rely on configured providers.
- Object storage/upload provider behavior depends on environment configuration.

Chapter 3: Stakeholder and Role Analysis

3.1 Customer

The customer is the buyer role. Customers use the marketplace to discover products, compare options, place orders, track delivery, request returns, review products, and contact support.

Customer priorities:

- Trustworthy product information.
- Clear prices and discounts.
- Smooth checkout.
- Accurate order tracking.
- Easy returns and support.
- Useful recommendations and alerts.

3.2 Vendor

The vendor is the seller role. Vendors use Amiyo-Go as a seller center to manage shop identity, products, stock, orders, packing, dispatch, returns, finance, promotions, reviews, and support communication.

Vendor priorities:

- Clear onboarding and KYC status.
- Product approval workflow.
- Fast order handling.
- Finance clarity.
- Campaign and voucher tools.
- Shop reputation and customer communication.

3.3 Admin

The admin is the marketplace operator role. Admins control the platform through queues, policies, settings, staff permissions, audit logs, moderation, logistics, finance, trust, and analytics.

Admin priorities:

- Central visibility.
- Exception handling.
- Approval workflows.
- Fraud and dispute control.
- Revenue and payout accuracy.
- Staff access governance.
- Operational dashboards.

Chapter 4: Technology Stack

4.1 Frontend Stack

```
| Area | Technology |
| Framework | React |
| Router | React Router |
| Build tool | Vite |
| Styling | Tailwind CSS and reusable UI components |
| Icons | lucide-react |
| HTTP | axios and fetch patterns |
| Forms | react-hook-form where used |
| Charts | Chart.js, Recharts |
| Maps | Leaflet and react-leaflet |
| Notifications | react-hot-toast, push subscription UI |
| i18n | i18next and react-i18next |
| PWA | Manifest and service worker support |
| Tests | Jest, React Testing Library |
```

4.2 Backend Stack

```
| Area | Technology |
| Runtime | Node.js |
| API framework | Express |
| Database | MongoDB |
| ODM | Mongoose |
| Native database access | mongodb package where needed |
| Auth integration | Firebase Admin SDK |
| File upload | multer, upload service, storage adapters |
| PDF generation | pdfkit |
| Queues | BullMQ and Redis where configured |
| Email | nodemailer and email services |
| Push | web-push |
| Payments | Provider adapter services including bKash/Nagad/SSLCommerz/Stripe patterns |
| Courier | Courier provider service and Amiyo Delivery integration service |
| Security | helmet, cors, rate limit, sanitization |
| Logging | winston |
| Tests | Jest, Supertest |
```

4.3 Infrastructure and Integrations

External or optional integrations include:

- Firebase authentication and admin claims.
- MongoDB Atlas or local MongoDB.
- Redis for queues where enabled.
- Email provider.
- Web push provider keys.
- Courier providers such as RedX, Steadfast, local/internal delivery, and Amiyo Delivery integration.
- Payment gateways and manual payment verification.
- Object storage adapters.

Chapter 5: System Architecture

5.1 Architectural Style

Amiyo-Go uses a modular monolith architecture. All business domains are inside one backend application, but each major domain has its own routes, controllers, models, services, utilities, and frontend pages.

This architecture is appropriate because:

- It is easier to develop and test than microservices.
- It keeps cross-domain workflows in one codebase.
- It allows future extraction of heavy modules if scale demands it.
- It supports fast iteration for marketplace features.

5.2 High-Level System Flow

```
Customer Web App / Vendor Dashboard / Admin Dashboard
-> React frontend with role-based layouts and guards
-> Express API
-> Domain route modules
-> Services and utilities
-> MongoDB collections/models
-> Optional queues, storage, courier, payment, email, push
```

5.3 Main Backend Domains

The backend registers route groups for:

- Products and catalog.
- Search and discovery.
- Categories and dynamic categories.
- Orders and vendor orders.
- Shipments and logistics.
- Users, accounts, addresses.
- Wishlist and alerts.
- Reviews and Q&A.
- Coupons, vouchers, offers, campaigns, promotions.
- Returns and refunds.
- Payments and webhooks.
- Support and chats.
- Flash sales and recommendations.
- Growth systems.
- Trust and safety.
- Platform settings.
- Delivery settings and courier rules.
- Vendor staff, KYC, shop, products, logistics, finance, growth.
- Admin users, vendors, products, payments, banners, settings, staff, COD, reviews, finance, payouts, analytics, logistics, customers, platform, audit.

5.4 Frontend Role Layouts

The frontend separates role experience through:

- Main/customer layout.
- Vendor layout.
- Admin layout.
- Route guards.
- Role-specific navigation.
- Protected University pages for vendor/admin training.

5.5 Screenshot Walkthrough and How It Works

This screenshot set was captured from the running frontend on 2026-06-04. It intentionally uses public and access pages only, so the PDF does not expose private admin/vendor data or real customer records. Protected vendor/admin dashboards are documented through workflows and route descriptions instead of unauthenticated screenshots.

˜ù‡ ›‰Í™YÍ› ¿š‰¶™ù—œ ²œ² èžièžÂÙæžÂÙæž¢ ¤›é°›ù–œr °›é¨›ù, frontend šYÇ™YÇ š‰Ç™9¯›É¾ ›™¯›ÉÇ™¹Ç–
vendor/admin dashboard ›‰Í™YÍ› ¿š‰¶™ùÇ› ¬ši²œr porkflow ˜ù¬˜" &÷WFR FW67 iption ši¿šù¼œr ¬œÙ¯›é–œÙ¯›

Figure 1: Customer Home Page

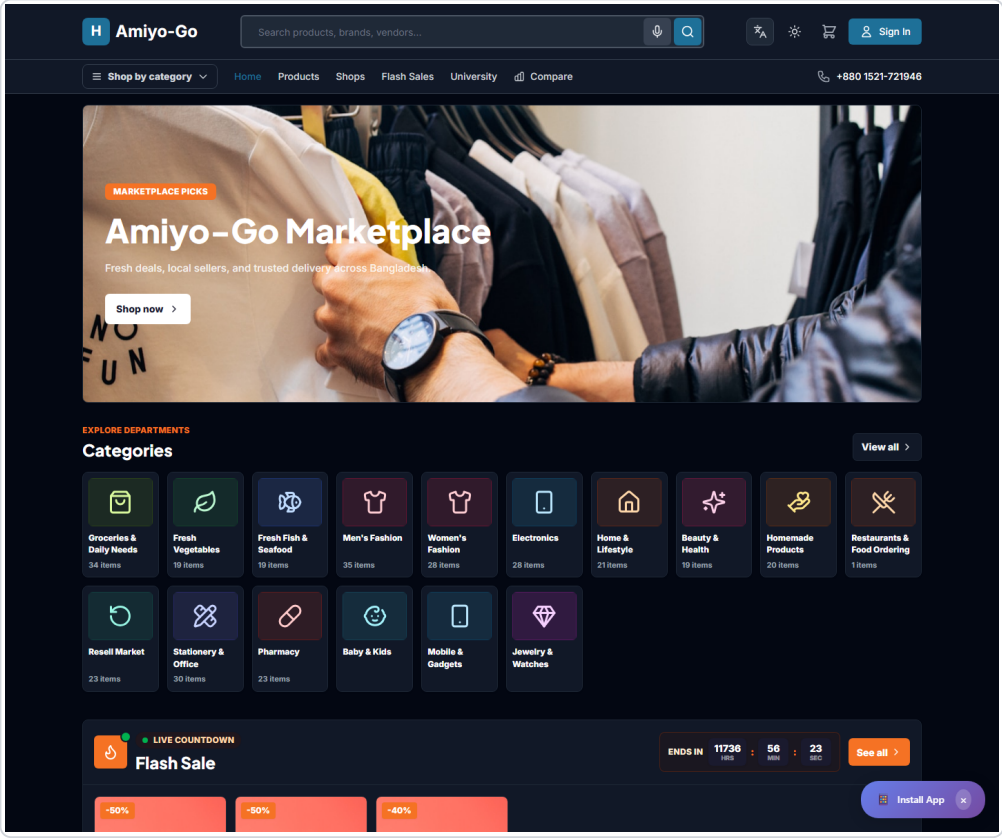

Figure: Customer home page with modern marketplace sections

How it works in English:

- The customer enters through the home page.
- The header gives search, category navigation, product links, shops, flash sales, compare, language, theme, cart, and sign-in access.
- The hero area promotes the marketplace and sends users into shopping.
- Category, flash sale, product, and shop sections help users discover products quickly.
- Admin-controlled banners and sections can change the front page without code changes.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- ™Y¾›‰Í™ù®›é° home page ši¿šù¼œr Ö ketplace-˜ò ªœÙ°šÉÇ›b •› Ç–@
- Header šYÇ™YÇ search, category, product, shops, flash sale, compare, language, theme, cart ˜ù¬˜" 6–vâÖ–â 66W72 ª
- Hero section marketplace offer šiÇ™i¾šù¼ ˜ù¬˜" W6W"Ù•œr 6†÷ –ær fÆðw-šIÇ š‰¿šù¼œr ¯›é¯›Éd
- Category, flash sale, product ˜ù¬˜" 6†÷ 6V7F–öâ ¦œÙ°œ ¤ product discovery ™Y°šIÇ ›‰¾›™¾šùÍšò •› Ç–@
- Admin-controlled banner ˜ù¬˜" 6V7F–öâ 6öFR 6† ævR ››é¡›É¾˜r †öÖW vR W F FR •› ¤œr ª›é°œeyd

Figure 2: Customer Learning Center

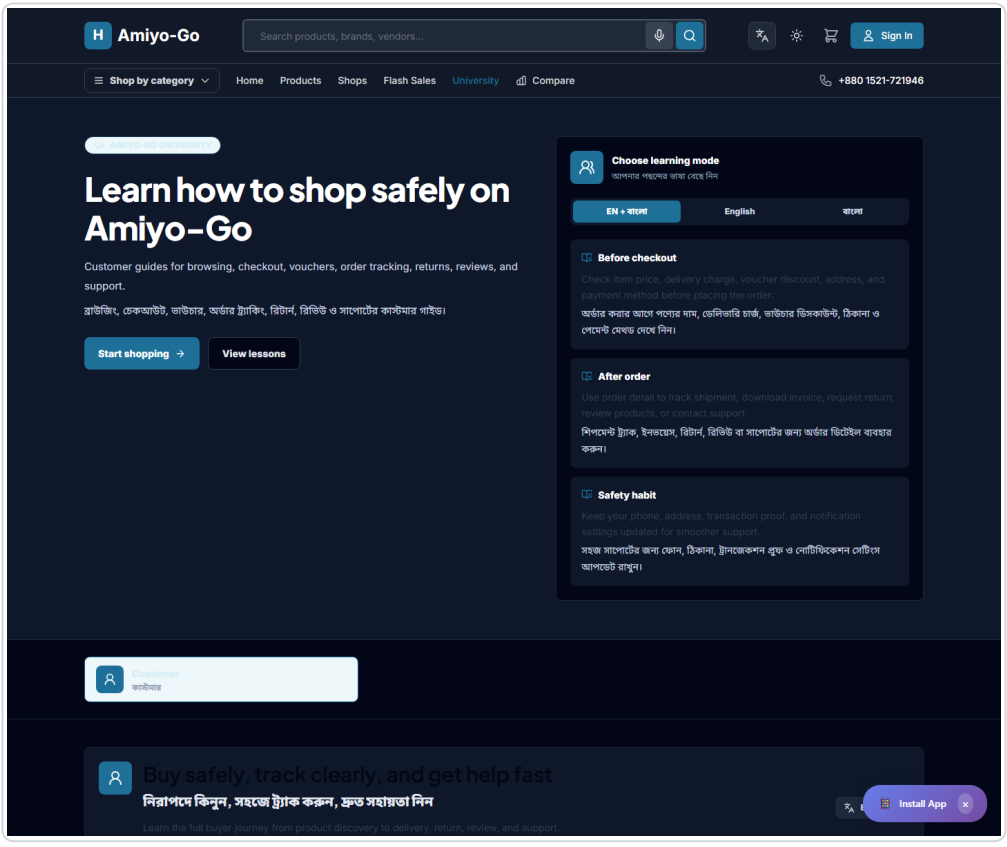

Figure: Customer-only Amiyo-Go University page

How it works in English:

- Public users only see customer learning content on `/university`.
- Vendor and admin operating lessons are not bundled into the public route.
- Customers learn checkout, vouchers, order tracking, returns, reviews, and support.
- Language mode supports English, Bangla, or both together.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- Public user `/university` š©Ç™ÉÇ ›iÁšyÁ customer guide šiÇ™iÇ–@
- Vendor/admin operating lesson public route-˜ò undle ™Y°›â ¹šù¼ š‰¾–@
- Customer checkout, voucher, order tracking, return, review ˜ù¬˜" 7W ÷ t ›iÇ™iÇ–@
- Language mode English, Bangla šÉ¾ šiÁ˜r ›é·›â •™Y¸›é¥œr 7W ÷ t ™Y°œeyd

Figure 3: Shop Discovery

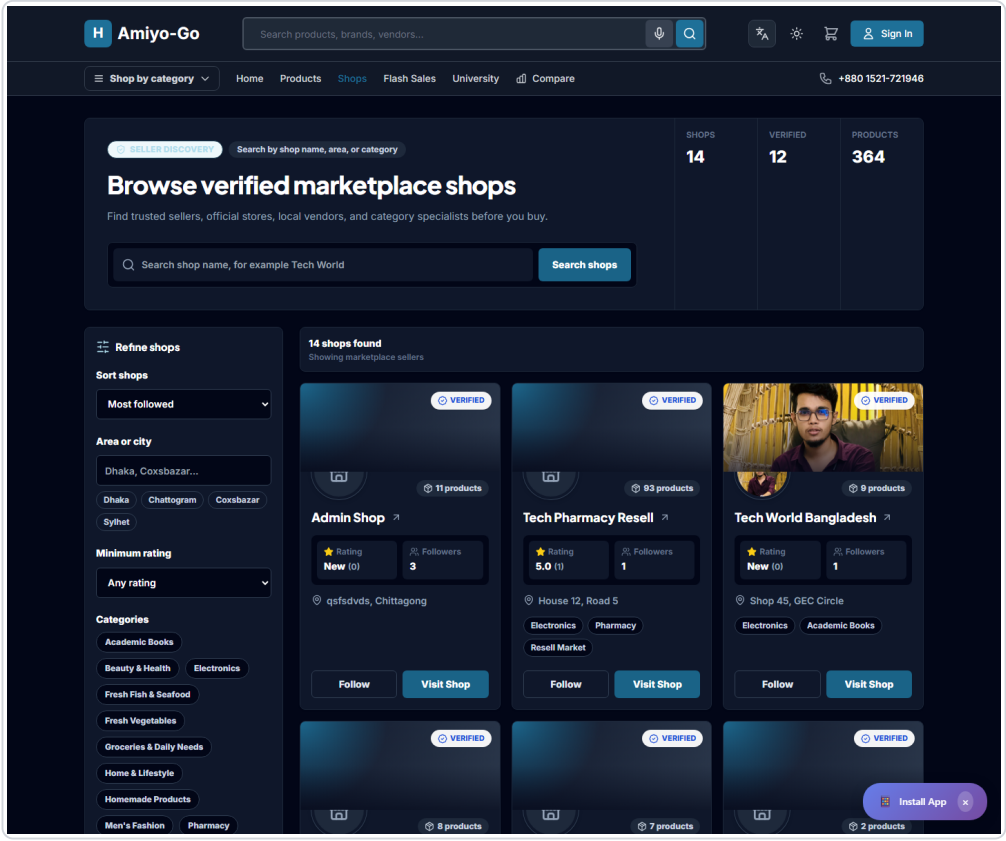

Figure: Shop discovery page with filters, shop cards, and follow action

How it works in English:

- Customers can browse verified marketplace shops.
- Search supports shop name, area, and category intent.
- Filters allow sorting by popularity, location, rating, and category.
- Each shop card shows trust signals, product count, follower count, location, and visit/follow actions.
- The backend `/api/shops` route returns safe public vendor fields only.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- Customer verified marketplace shop browse ™Y°šIÇ š©¾› Ç–@
- Search shop name, area ˜ù¬˜" 6 FVv÷ y intent support ™Y°œyd
- Filter ši¿šù¼œr ÷ VÆ ity, location, rating ˜ù¬˜" 6 FVv÷ y ˜Y¨œ ¯›é¯›ÉÀ result šiÇ™i¾ šù¾šù¼–@
- Shop card-˜ò G ust signal, product count, follower count, location, visit ˜ù¬˜" `ollow action šY¾™YÇ–@
- Backend `/api/shops` route ›iÁšyÁ safe public vendor field return ™Y°œyd

Figure 4: Account Registration and Address Capture

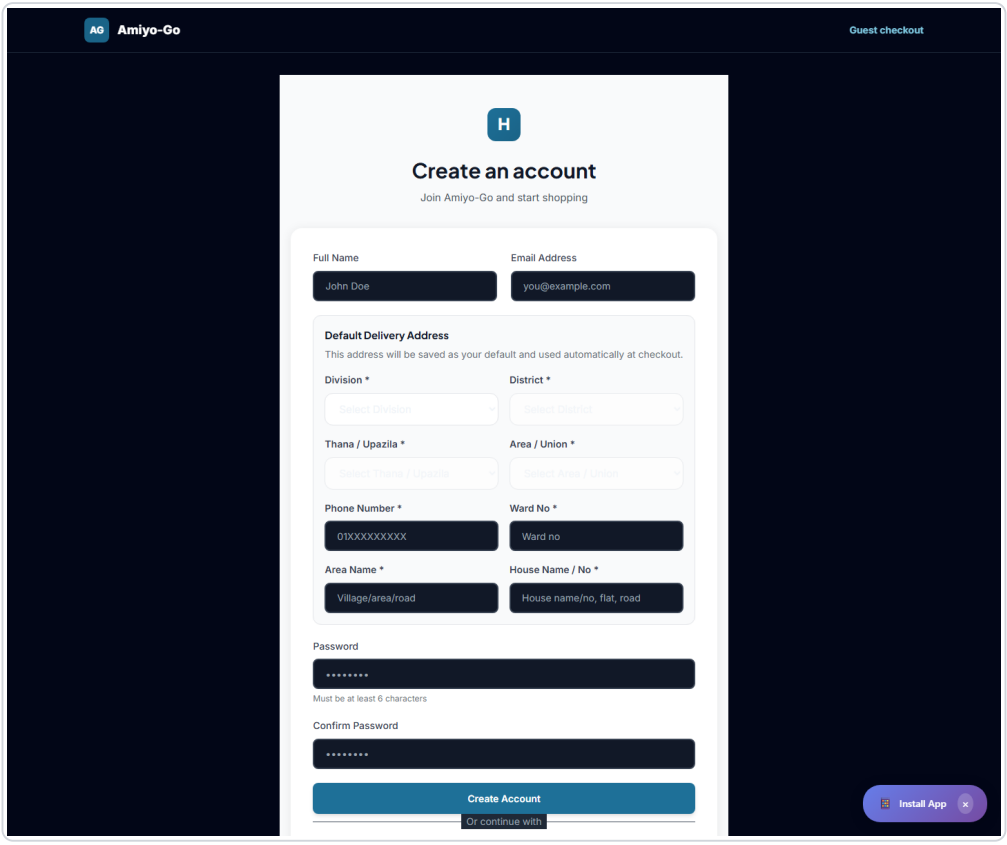

Figure: Customer registration page with default address fields

How it works in English:

- A new customer creates an account before using full account features.
- Registration captures basic identity and default delivery address.
- Address fields are structured for Bangladesh delivery workflow.
- The saved address later supports checkout, delivery calculation, courier assignment, and support.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- š‰¤œ ¨ customer full account feature šÉÍšù¬›™¾› Ç› †™yÇ account šIÈ› ¿ ™Y°œyd
- Registration basic identity ˜ù¬˜" FV`ault delivery address š‰Ç•ùd
- Address fields Bangladesh delivery workflow ˜Y¨œ ¯›é¯›ÉÀ structured–@
- Saved address š©°œr 6†V0kout, delivery calculation, courier assignment ˜ù¬˜" 7W ÷ t-˜ò ¬œÙ¯šÉ¹›é° ›™¯›Éd

Figure 5: Login and Protected Access

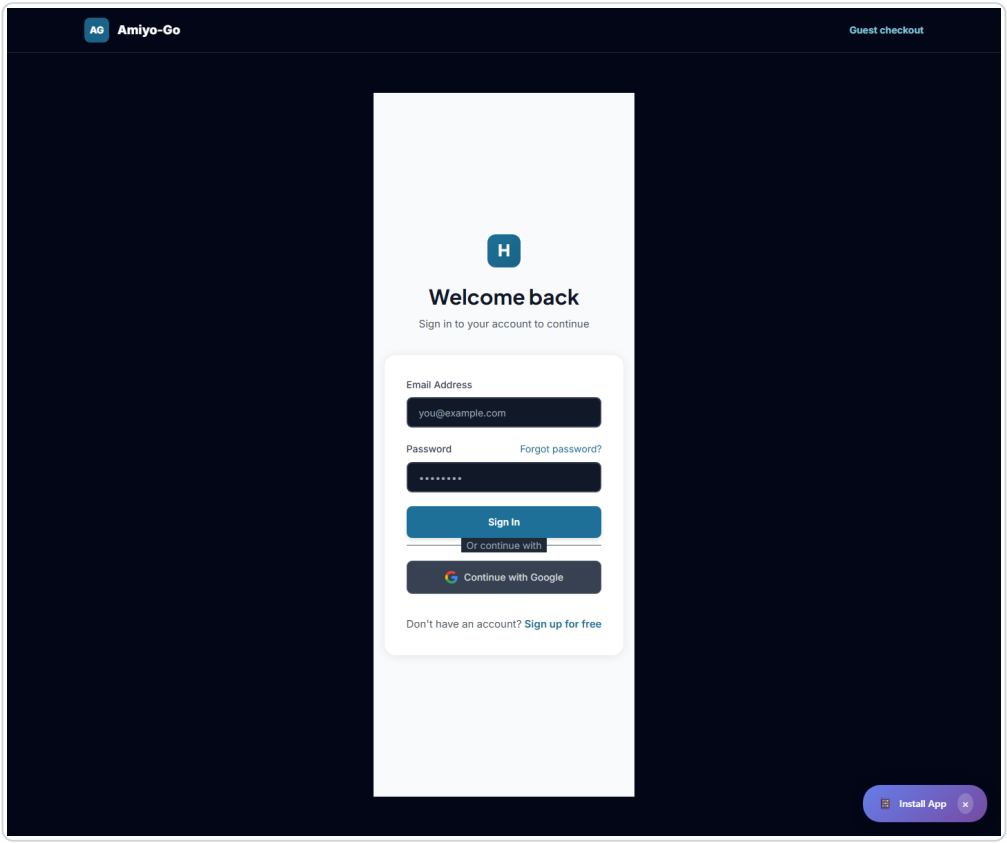

Figure: Login page for customer, vendor, and admin access

How it works in English:

- Users sign in before accessing protected customer, vendor, or admin pages.
- Route guards redirect unauthenticated users to login.
- Admin and vendor pages also check role/permission after authentication.
- Guest checkout remains available where the project allows it.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- Protected customer, vendor šÉ¾ admin page access ™Y°›é° ˜i—œr W6W" 6–vâ –â •› Ç–@
- Route guard unauthenticated user-™YÇ login page-˜ò ª›é ›é¯›Éd
- Admin ˜ù¬˜" `endor page authentication-˜ù° š©°œr &öÆR÷ W mission check ™Y°œyd
- Project šùÇ™i¾š‰Ç allow ™Y°œr ¸œy–›é¨œr wVW7B 6†V0kout available šY¾™YÇ–@

Figure 6: Mobile Home Experience

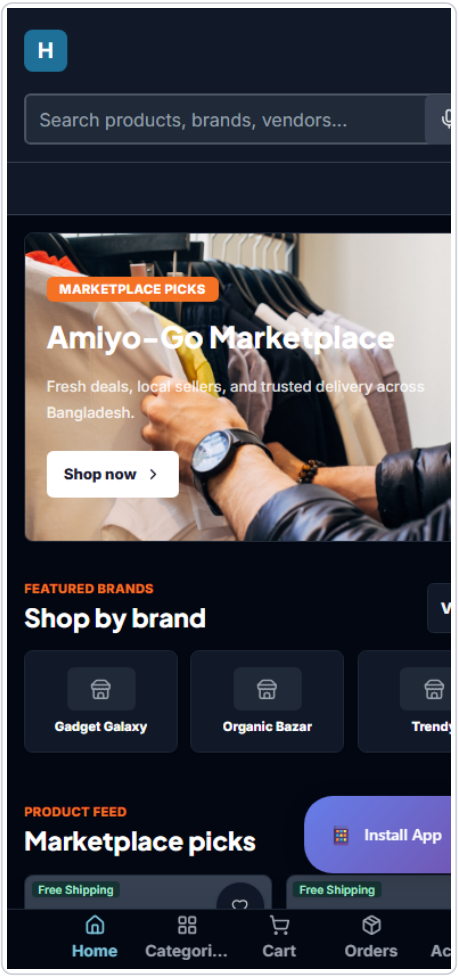

Figure: Responsive mobile home page

How it works in English:

- The same marketplace experience adapts to a mobile viewport.
- Navigation, category discovery, cart, and account access remain reachable.
- PWA support allows the site to behave closer to an installable app.
- Mobile layout is important because marketplace traffic is usually phone-heavy.

**™YÀšÙ¾šÉÇ ™Y¾™Â •› Ç šÉ¾˜)²›é¯›Ã¢¢

- ˜ù•˜r Ö ketplace experience mobile viewport ˜Y¨œ ¯›é¯›ÉÀ responsive ›™¯›Éd
- Navigation, category discovery, cart ˜ù¬˜" 66÷VçB 66W72 ¸›™œœr ª›é“šù¼›â ¯›é¯›Éd
- PWA support site-™YÇ installable app-˜ù° šé¤œ² ¬œÙ¯šÉ¹›é°šùË™yÍšò •› Ç–@
- Marketplace traffic ›‰¾šy¾› £šB Öö&–ÆRÖ†V vy ›™“šù¼›é¯›Â Öö&–ÆR Æ yout ™iÁšÂ —œ °œ ¤œÙ¬š©Â› Íš9d

Chapter 6: Frontend Architecture

6.1 Public and Customer Pages

Important customer-facing pages include:

- Home.
- Products.
- Product detail.
- Category page.
- Search results.
- Shops listing.
- Shop detail.
- Cart.
- Checkout and guest checkout.
- Order confirmation.
- Orders.
- Order detail.
- Returns.
- Wishlist.
- Compare.
- Loyalty dashboard.
- Notifications.
- Messages/support.
- My reviews.
- Profile and addresses.
- Flash sales.
- Campaign landing.
- University customer learning page.

6.2 Vendor Pages

Important vendor pages include:

- Vendor dashboard.
- Vendor KYC.
- Vendor shop settings.
- Vendor shop profile.
- Vendor bank settings.
- Vendor category requests.
- Vendor products.
- Add/edit product.
- Bulk upload.
- Product detail.
- Orders and order detail.
- Returns and return detail.
- Finance.
- Marketing.

- Reports.
- Reviews.
- Q&A.
- Messages/support chat.
- Protected Vendor University.

6.3 Admin Pages

Important admin pages include:

- Admin dashboard.
- Operations center.
- Vendors and vendor KYC.
- Vendor chats.
- Products and product moderation.
- Category management.
- Dynamic categories.
- Category requests.
- Orders.
- Returns.
- Payment verifications.
- Finance and payouts.
- COD delivery/reconciliation.
- Logistics.
- Delivery settings.
- Promotions, offers, coupons, vouchers, flash sales.
- Banners and home controls.
- Newsletter.
- Customers and customer insights.
- Reviews and Q&A moderation.
- Support.
- Trust and safety.
- User/staff management.
- Platform controls/settings.
- Audit logs.
- Analytics reports.
- Protected Admin University.

6.4 UI and Styling Pattern

The frontend uses Tailwind CSS with reusable components. UI patterns include:

- Cards for repeated items.
- Data tables for admin and vendor queues.
- Status badges for workflow states.
- Filter bars and saved views.
- Mobile bottom navigation.
- Professional home page sections.
- Product cards with badges and vendor links.
- Shop storefront headers, tabs, maps, and product grids.
- Dashboard metric cards and queue cards.

6.5 State and Data Access Pattern

The project primarily uses React state, context-style utilities, axios/fetch API calls, and route-level data loading patterns. Authentication state is integrated with Firebase and role guards. Some pages use local state and effect-driven fetching, while service modules centralize API calls in several domains.

Chapter 7: Backend Architecture

7.1 API Layer

The backend API is organized through Express route modules under `Server/routes`. `Server/index.js` registers the domain routes and applies middleware such as:

- Helmet security headers.
- CORS configuration.
- JSON body limits.
- Request sanitization.
- Rate limits for API, search, uploads, product views, and payments.
- Static upload serving.
- Audit middleware for sensitive actions.
- Error handling.

7.2 Service Layer

Service modules under `Server/services` handle reusable business logic. Important services include:

- Marketplace event bus.
- Order event service.
- Notification services.
- Email and push services.
- Invoice generation.
- Promotion service.
- Promotion rules engine.
- Discount calculator service.
- Loyalty service.
- Recommendation service.
- Growth event bus and growth notification service.
- Analytics event and warehouse services.
- Trust policy, risk scoring, trust case, and enforcement services.
- Courier provider service.
- Amiyo Delivery integration service.
- Payment provider services.
- Search provider registry and Mongo search provider.
- Upload and storage services.
- Bulk upload queue.

7.3 Utility Layer

Utilities support cross-cutting logic:

- Logistics state machine.
- Logistics scope helpers.
- Delivery calculator.

- Customer order experience.
- Manual payment proof helpers.
- Vendor settlement.
- Vendor staff audit.
- Platform features.
- Geocoding with Nominatim.
- Query optimizer.

7.4 Model Layer

The backend model layer uses Mongoose schemas for major collections:

- User.
- Vendor.
- VendorStaff.
- VendorShop.
- Product.
- Category.
- DynamicProduct.
- DynamicCategory.
- Order.
- VendorOrder.
- Shipment.
- Return.
- Payment.
- PaymentVerification.
- Coupon.
- Offer.
- Promotion.
- Campaign.
- FlashSale.
- Review.
- Question.
- SupportTicket.
- Notification.
- Loyalty.
- Wishlist.
- StockAlert.
- VendorPayout.
- TrustSafety.
- AuditLog.
- AnalyticsSummary.
- Banner.
- DeliverySettings.
- StoreLocation.

Chapter 8: Database and Persistence Model

8.1 Current Database

The current data store is MongoDB. The project uses both Mongoose models and native MongoDB access in places. This is important because some target architecture diagrams may mention PostgreSQL as a future migration target, but the current implementation is MongoDB.

8.2 Core Persistence Groups

```
| Domain | Main models/collections |
| Identity | User, Firebase identity claims, staff permissions |
| Vendor | Vendor, VendorShop, VendorStaff, VendorKYC fields |
| Catalog | Product, Category, DynamicCategory, DynamicProduct |
| Cart/Wishlist | Frontend cart state, Wishlist, StockAlert |
| Checkout/Orders | Order, VendorOrder, OrderEvent |
| Payments | Payment, PaymentVerification, provider transaction fields |
| Logistics | Shipment, delivery settings, dispatch assignments, courier metadata |
| Returns | Return, reverse logistics fields, refund state |
| Reviews/Q&A | Review, Question |
| Support | SupportTicket, LiveChat, VendorChat, AdminVendorChat |
| Promotions | Coupon, Voucher routes, Offer, Promotion, Campaign, FlashSale |
| Loyalty | Loyalty wallet/transactions and rewards |
| Notifications | Notification, NotificationSubscription, PushSubscription |
| Trust | TrustSafety, risk events, reports, disputes, enforcements, appeals patterns |
| Analytics | AnalyticsSummary, event stream services, campaign analytics |
| Admin/Audit | AuditLog, Permission, platform settings |
```

8.3 Data Principles

The project follows these data principles:

- Store order snapshots for price, discount, delivery fee, and promotion logic.
- Split master orders into vendor orders for seller-center workflows.
- Preserve payment and manual verification state separately from order display.
- Keep shipment state machine fields separate from reverse logistics and COD state.
- Use audit logs for sensitive admin actions.
- Store promotion snapshots so historical orders remain accurate after rules change.
- Separate public shop data from sensitive vendor KYC/bank data.

Chapter 9: Feature Documentation by Role

9.1 Customer Features

9.1.1 Account and Identity

Customers can:

- Register and login.
- Logout.
- Reset password.
- Manage profile.
- Use guest checkout where enabled.
- Manage saved addresses.
- Set default delivery address.
- Configure notification preferences.
- Request account export/delete through account routes.

9.1.2 Discovery and Shopping

Customers can:

- Browse home page sections.
- Browse categories.
- Search products.
- Use filters and sorting.
- View product detail.
- View vendor storefronts.
- Browse all shops.
- Follow shops.
- Use wishlist.
- Compare products.
- View recently viewed and recommendation surfaces.
- Access flash sales and campaign pages.

9.1.3 Product Detail

Product detail supports:

- Media gallery.
- Variant selection.
- Product video support where media URL exists.
- Badges and trust signals.
- Delivery estimate widget.
- Product Q&A.
- Reviews with rich review components.
- Seller info strip.
- Share/report actions.

- Frequently bought together.
- Product recommendations.
- Stock alert button.

9.1.4 Cart and Checkout

Checkout supports:

- Cart item management.
- Vendor-grouped cart behavior where implemented.
- Coupon/voucher validation.
- Loyalty points redemption.
- Shipping/delivery fee display.
- Address selection.
- Payment method selection.
- COD and manual/online payment patterns.
- Guest checkout.
- Discount persistence into order, invoice, and order detail.

9.1.5 Orders and Post-Purchase

Customers can:

- View order history.
- Open order detail.
- Track shipment.
- See courier assignment when available.
- See item subtotal, delivery charge, discount, and final total.
- Reorder.
- Download or view invoice where enabled.
- Request return/refund.
- Review purchased products.
- Open support.

9.1.6 Engagement

Customer engagement includes:

- Notifications.
- Push subscriptions.
- Wishlist alerts.
- Stock alerts.
- Loyalty dashboard.
- Rewards.
- Followed vendor signals.
- Amiyo-Go University customer guide.

9.2 Vendor Features

9.2.1 Onboarding

Vendors can:

- Register as seller.
- Submit KYC.
- Manage shop profile.
- Add pickup/contact information.
- Request category access.
- See vendor status.
- Use protected vendor dashboard after approval.

9.2.2 Shop Management

Vendor shop tools include:

- Shop settings.
- Shop media.
- Shop location.
- Public storefront data.
- Slug-based shop URLs.
- Banner/logo.
- Tagline and description.
- Policies.
- Working hours.
- Social links.
- Vendor categories.

9.2.3 Product and Inventory

Vendors can:

- Add products.
- Edit products.
- Save product fields and variants.
- Upload product images.
- Submit for moderation.
- View rejected/pending/approved states.
- Use bulk upload jobs.
- Track product detail and product performance.
- Handle stock and low stock workflows.

9.2.4 Orders and Fulfillment

Vendors can:

- View vendor order queues.
- Open order details.
- Process orders.
- Add notes where supported.
- Pack items.
- Print packing slips/labels where workflow supports it.
- Mark pickup-ready.
- Work with logistics and courier assignment.
- Handle returns and disputes.

9.2.5 Finance

Vendor finance includes:

- Earnings dashboard.
- Commission visibility.
- Refund and deduction visibility.
- COD settlement visibility.
- Payout request and history.
- Bank settings.
- Finance ledger style views.

9.2.6 Marketing and Reputation

Vendors can:

- Manage marketing/vouchers.
- Join or participate in campaigns where enabled.
- Track reports.
- View reviews.
- Reply or manage reputation workflows where implemented.
- Use Q&A.
- Use vendor support chat.
- Learn through protected Vendor University.

9.3 Admin Features

9.3.1 Platform Control

Admins can:

- Access admin dashboard.
- Use RBAC-aware admin navigation.
- Manage staff.
- Use platform settings.
- View audit logs.
- Use admin operations center.
- Use admin global search.
- Control home banners and sections.

9.3.2 Vendor Management

Admins can:

- Review vendor applications.
- Review KYC.
- Approve/reject/suspend vendors.
- Review vendor details.
- Monitor vendor performance.
- Manage vendor categories.
- Review vendor staff and access patterns.

9.3.3 Product and Catalog Operations

Admins can:

- View all products.
- Moderate pending products.
- Approve/reject/disable products.
- Bulk moderate products.
- Scan moderation config.
- Review duplicates.
- Review IP/counterfeit reports.
- Manage categories and dynamic categories.
- Manage category fields/attributes.

9.3.4 Orders and Returns

Admins can:

- View and search global orders.
- Export orders.
- View returns queue.
- Review return evidence.
- Approve/reject/escalate returns.
- Process refund states through finance routes.
- Monitor delivery failures and logistics exceptions.

9.3.5 Finance and Payouts

Admins can:

- Verify manual payments.
- Approve/reject payment proof.
- Use payment verification bulk actions.
- View finance operations.
- Manage payout queue.
- Approve/reject/mark payout paid.
- Manage commission rules.
- View refund workflow.
- Export revenue reports.
- Reconcile COD.
- Monitor payout exposure.

9.3.6 Logistics

Admins can:

- View logistics dashboard.
- View state machine.
- List shipments.
- Assign courier.
- Update shipment state.
- Record delivery attempts.
- Mark return-to-origin.
- Confirm RTO received.
- Generate labels.
- Confirm manifest pickup.

- Download waybill.
- Manage COD state.
- Manage delivery zones.
- Manage courier partners.
- Manage dispatch manifest.
- Manage pickup staff.
- Manage delivery fee rules.
- Monitor failed deliveries.

9.3.7 Growth and Promotions

Admins can:

- Manage coupons/vouchers.
- Manage offers.
- Manage promotions.
- Pause/resume/expire/duplicate promotions.
- Manage flash sales.
- Manage campaigns.
- Manage banners.
- Manage notification templates/logs.
- Run abandoned cart detection.
- Review growth analytics.
- Manage experiments where enabled.

9.3.8 Trust and Safety

Admins can:

- Review trust and safety dashboard.
- Process reports.
- Review disputes.
- Apply enforcement actions.
- Handle appeals patterns.
- Review suspicious reviews, returns, vendors, payouts, and promotions.
- Use risk scoring and trust policy services.

9.3.9 Analytics and Reporting

Admins can:

- View analytics reports.
- Use KPI framework and event services.
- Review growth analytics.
- Review customer insights.
- Export reports.
- Track logistics, finance, trust, growth, campaign, and operational metrics.

Chapter 10: Business Workflows

10.1 Customer Purchase Workflow

```
Customer visits home
-> browses category/search/shop/product
-> opens product detail
-> selects variant
-> adds to cart
-> applies voucher/points if available
-> selects address
-> selects payment method
-> reviews order total
-> places order
-> order is created
-> vendor order split is created
-> payment/COD state is recorded
-> shipment draft or logistics state is prepared
-> customer tracks order
-> customer receives item
-> review/return/support is possible
```

10.2 Discount Persistence Workflow

```
Checkout validates coupon or promotion
-> backend recalculates discount on order placement
-> order stores discount breakdown and final total
-> invoice reads stored discounted totals
-> order list/detail displays payable amount
-> vendor/admin views use stored order snapshot
```

This workflow prevents the mismatch where checkout shows a discount but order or invoice shows the original price.

10.3 Vendor Product Workflow

```
Vendor creates product
-> saves product information, category, images, price, stock
-> submits product for moderation
-> admin reviews listing
-> approved product becomes public
-> rejected product returns with reason
-> vendor edits and resubmits if needed
```

10.4 Vendor Fulfillment Workflow

```
Customer places order
-> vendor receives vendor order
-> vendor verifies item/payment/address
-> vendor packs item
-> vendor marks pickup-ready
-> admin/vendor courier workflow starts
-> shipment moves through logistics state machine
```

```
-> delivered, failed, or RTO state is recorded
-> finance settlement follows payment and delivery state
```

10.5 Admin Exception Workflow

```
Order/vendor/product/return/payment/logistics issue enters queue
-> admin filters and assigns case
-> admin reviews evidence and linked records
-> admin takes allowed action
-> audit log records action
-> notification or workflow state updates affected roles
```

10.6 Return Workflow

```
Customer requests return
-> vendor/admin reviews request and evidence
-> return approved/rejected/escalated
-> reverse logistics is scheduled if approved
-> returned item is received
-> item is inspected
-> restock/dispose/refurbish decision is recorded
-> refund and vendor deduction are applied
```

10.7 COD Workflow

```
COD pending
-> courier collects cash at delivery
-> COD collected
-> courier/platform remits cash
-> COD remitted
-> platform settles vendor after deductions
```

Exception states include COD failed and COD disputed.

10.8 Protected University Workflow

```
Public user opens /university
-> sees customer-only learning content
Vendor opens /vendor/university
-> route guard protects seller learning content
Admin opens /admin/university
-> admin permission guard protects operator learning content
```

This prevents public users from viewing internal vendor/admin operating guidance.

10.9 End-to-End Operating Narrative in English

Amiyo-Go works as a connected marketplace loop. The customer starts from the home page, search, categories, or shops. After choosing a product, the customer checks price, stock, seller information, delivery estimate, reviews, Q&A, and available offers. The customer adds items to cart, applies vouchers or loyalty points, selects an address and payment method, then places the order.

When the order is placed, the backend recalculates totals and stores a final order snapshot. This snapshot includes subtotal, delivery charge, discount, payable total, promotion details, payment method, customer

address, and item information. The master order is split into vendor orders so each seller can process only their part of the purchase.

The vendor receives the vendor order in the seller center. The vendor checks item, stock, payment state, and delivery address. The vendor packs the product, prepares slip/label where available, and marks the order pickup-ready. Admin or vendor logistics workflows then assign courier, schedule pickup, update shipment state, and track delivery.

If the order is COD, COD state runs beside shipment state. If delivery succeeds, COD can move from pending to collected, remitted, and settled. If delivery fails, admin can record delivery attempts, re-attempt delivery, or mark return-to-origin. If the customer requests a return after delivery, reverse logistics starts and the item moves through approval, pickup, received, inspection, and final stock/refund decision.

The admin controls the marketplace by queues. Admins review vendor applications, product moderation, payments, returns, payouts, logistics exceptions, COD reconciliation, support tickets, trust cases, campaigns, staff permissions, analytics, and audit logs. This queue-first model keeps customer, vendor, and platform operations connected and traceable.

10.10 ›‰®œÙªœ)°œÙ£ ™Y¾™ÉÇ› §›é°›â ¬›é,›)¾šù¼

Amiyo-Go ˜ù•™ù¿ connected marketplace loop ›™¿›‰ÇšÉÇ ™Y¾™Â •› Ç–B 7W7FöÖW" †öÖR vP, search, category šÉ¾™1¾˜r •› ¾› ª› Ç customer price, stock, seller information, delivery estimate, review, Q&A ˜ù¬˜" vailable offer šiÇ™i voucher šÉ¾ loyalty point apply ™Y°œrÂ FG&W72 “ payment method select ™Y°œr ÷&FW" Æ 6R •› Ç–@

Order place ›™²œr & 0kend ˜i¬›é° total calculate ™Y°œr f–æ Â ÷&FW" 6æ 6†÷B 6 ve ™Y°œyd ˜ù‡ snapshot-˜ò 7V'F÷F discount, payable total, promotion details, payment method, customer address ˜ù¬˜" —FVÒ –æ`ormation šY¾™YÇ–B Ö order vendor order-˜ò 7 Æ—B ¹šù¼, šù¾šIÇ š©Í› ¤›ùŸ›ò 6VÆÆW" ¶œ §œ ¤›é° š‰¿™ÉÇ› ÷&FW" …˜)¶ process ™Y°

Vendor seller center-˜ò `endor order š©¾šù¼–B Vendor item, stock, payment state ˜ù¬˜" FVÆ—`ery address check ™Y° pack ™Y°œrÂ 6Æ— öÆ &VÂ &V G’ •› Ç ˜ù¬˜" ÷&FW" –0kup-ready mark ™Y°œyd šI¾› ª› FÖ–â ¬›â `endor logistics w schedule ™Y°œrÂ 6†— ÖVçB 7F FR W F FR •› Ç ˜ù¬˜" FVÆ—`ery track ™Y°œyd

Order šù¦›ò 4ôB ¹šù¼, šI¾›™²œr 6†— ÖVçB 7F FRÙ•› ª›é¶›éª›é¶›ò 4ôB 7F FR š›)Ç–B FVÆ—`ery successful ›™²œr 4 state-˜ò ¯œy¤œr ª›é°œyd Delivery failed ›™²œr  FÖ–â FVÆ—`ery attempt record ™Y°œrÂ &RÖ GFV× B ¦›ù¤œr ª›é°œr customer return request ™Y°›)Ç reverse logistics ›iÁ› Á ›™¯›Â •šÉ, item approval, pickup, received, inspection ™2 f–æ Â decision-˜ù° šé§œÙ¯ ši¿šù¼œr ¯›é¯›Éd

Admin marketplace queue-based control ™Y°œyd Admin vendor application, product moderation, payment verification, return, payout, logistics exception, COD reconciliation, support ticket, trust case, campaign, staff permission, analytics ˜ù¬˜" VF—B Æör Ö æ vR •› Ç–B •˜r VWVRÖf—'7B ÖöFVÂ 7W7FöÖW , vendor ˜ù¬˜" Æ F`orm op ˜ù¬˜" G aceable › ¾™iÇ–@

Chapter 11: API and Service Map

11.1 Public and Customer APIs

Important API groups:

- `/api/products`
- `/api/search`
- `/api/categories`
- `/api/shops`
- `/api/orders`
- `/api/orders/guest`
- `/api/payments`
- `/api/shipments`
- `/api/returns`
- `/api/reviews`
- `/api/wishlist`
- `/api/addresses`
- `/api/coupons`
- `/api/vouchers`
- `/api/offers`
- `/api/flash-sales`
- `/api/recommendations`
- `/api/notifications`
- `/api/support`
- `/api/loyalty`
- `/api/stock-alerts`
- `/api/account`

11.2 Vendor APIs

Important vendor API groups:

- `/api/vendors`
- `/api/vendor/kyc`
- `/api/vendors/staff`
- `/api/vendor/shop`
- `/api/vendor/products`
- `/api/vendors/finance`
- `/api/vendors` for vendor order management routes.
- `/api/vendor/logistics`
- `/api/vendor/growth`
- `/api/vendor-chat`

11.3 Admin APIs

Important admin API groups:

- `/api/admin/dashboard`
- `/api/admin/users`
- `/api/admin/vendors`
- `/api/admin/vendors/kyc`
- `/api/admin/products`
- `/api/admin/payment-verification`
- `/api/admin/banners`
- `/api/admin/settings`
- `/api/admin/staff`
- `/api/admin/vouchers`
- `/api/admin/cod`
- `/api/admin/reviews`
- `/api/admin/orders`
- `/api/admin/vendor-marketing`
- `/api/admin/finance`
- `/api/admin/payouts`
- `/api/admin/alerts`
- `/api/admin/search`
- `/api/admin/promotions`
- `/api/admin/growth`
- `/api/admin/logistics`
- `/api/admin/customers`
- `/api/admin/trust-safety`
- `/api/admin/platform`
- `/api/admin/audit`
- `/api/admin/analytics`
- `/api/admin/dispatch`

11.4 Integration APIs and Services

Integration-related modules include:

- Payment provider services.
- Webhook routes.
- Courier provider service.
- Amiyo Delivery integration service.
- Upload service routes.
- Email queue and email service.
- Push service.
- SMS service placeholder.
- Analytics event ingestion.
- Marketplace event bus.

Chapter 12: Security, Trust, and Permissions

12.1 Authentication

Authentication is integrated with Firebase. Backend routes use token verification middleware and role checks such as admin/vendor/customer route protection.

12.2 Authorization

Authorization patterns include:

- AdminRoute and VendorRoute frontend guards.
- Backend `verifyToken`, `verifyAdmin`, and role/permission checks.
- Admin RBAC and permission-aware navigation.
- Staff permission controls.
- Protected vendor/admin University content.

12.3 API Protection

API protection includes:

- Helmet.
- CORS configuration.
- Rate limiting.
- Mongo sanitization.
- Upload limits.
- Payment/search/upload-specific rate limits.
- Audit middleware for sensitive admin operations.

12.4 Trust and Safety

Trust and safety modules support:

- Risk scoring.
- Reports.
- Disputes.
- Enforcement.
- Appeals pattern.
- Review moderation.
- Return abuse signals.
- Promo abuse protection.
- Payout risk controls.
- Auditability.

12.5 Staff Governance

Admin can add staff/managers and give section-based permissions. The intended production rule is:

- Staff can operate assigned sections.
- Staff should not receive delete/settings permissions unless necessary.
- Sensitive actions require audit logs.
- Super-admin-only permissions should remain restricted.

Chapter 13: Logistics and Courier Integration

13.1 Shipment State Machine

Forward logistics states:

```
created
-> pending_packing
-> packed
-> pickup_ready
-> pickup_scheduled
-> picked_up
-> in_transit
-> out_for_delivery
-> delivered
```

Exception states:

```
delivery_failed
-> out_for_delivery
or
delivery_failed
-> return_to_origin
```

13.2 Reverse Logistics State Machine

```
return_requested
-> return_approved
-> return_pickup_scheduled
-> return_picked_up
-> return_in_transit
-> return_received
-> inspected
-> restocked / disposed / refurbished
```

13.3 COD State Machine

```
cod_pending
-> cod_collected
-> cod_remitted
-> cod_settled
```

Exception states:

```
cod_failed
cod_disputed
```

13.4 Courier Assignment

Courier assignment can be:

- Internal/manual courier.
- Area-based courier rules.

- RedX adapter-backed booking when credentials are configured.
- Steadfast adapter-backed booking when credentials are configured.
- Amiyo Delivery integration.
- Future local instant delivery provider.

13.5 No-Paid-API Operating Mode

The marketplace can operate without paid courier APIs by using:

- Manual courier assignment.
- Internal courier state updates.
- Packing and pickup-ready workflow.
- Manual tracking number entry.
- Admin logistics dashboard.
- COD state updates.
- Manual delivery attempt/RTO handling.

This is important because the system can run before every delivery partner is integrated.

Chapter 14: Promotions, Growth, and Personalization

14.1 Promotion Engine

Promotion-related features include:

- Coupons.
- Vouchers.
- Offers.
- Platform promotions.
- Vendor marketing items.
- Flash sales.
- Campaigns.
- Promotion snapshots.
- Discount calculators.
- Stacking/conflict logic patterns.

14.2 Loyalty

Loyalty features include:

- Loyalty dashboard.
- Points redemption.
- Rewards.
- Customer loyalty ledger patterns.
- Admin loyalty adjustment.

14.3 Recommendations and Discovery

Discovery and recommendation features include:

- Product recommendations.
- Frequently bought together.
- Similar products.
- Recently viewed patterns.
- Search suggestions/history.
- Shop discovery.
- Followed vendor signals where enabled.

14.4 Notifications

Notification features include:

- In-app notification center.

- Push subscriptions.
- Email services.
- Growth notifications.
- Notification templates/logs in admin growth routes.
- Order/return/support/promotion notifications where emitted.

Chapter 15: Analytics and Reporting

15.1 Analytics Foundation

The project includes:

- Analytics event service.
- Analytics KPI framework.
- Analytics warehouse service.
- Analytics intelligence service.
- Growth event service.
- Campaign analytics service.
- Admin analytics routes.
- Vendor reports.

15.2 KPI Categories

Customer KPIs:

- Sessions.
- Product views.
- Add-to-cart rate.
- Checkout rate.
- Conversion rate.
- AOV.
- Repeat purchases.
- Notification engagement.

Vendor KPIs:

- GMV.
- Net sales.
- Order count.
- Fulfillment speed.
- Return rate.
- Cancellation rate.
- Stockout risk.
- Review score.
- Campaign GMV.

Admin KPIs:

- Total GMV.
- Commission revenue.
- Refund rate.
- Payout exposure.
- COD exposure.
- RTO rate.
- Support SLA.

- Logistics SLA.
- Fraud/dispute rate.
- Active customers/vendors.

15.3 Report Surfaces

Report surfaces include:

- Admin analytics reports.
- Admin dashboard overview.
- Admin operations overview.
- Finance reports and export.
- Vendor reports.
- Growth analytics.
- Campaign analytics.
- Customer insights.
- Logistics dashboard.
- Trust and safety dashboard.

Chapter 16: Testing and Quality Assurance

16.1 Existing Test Structure

The project contains frontend and backend test scripts:

- Client: `npm run test`
- Client: `npm run build`
- Server: `npm test`
- Server: `npm run test:all`
- Server: API and notification test scripts.

Frontend tests exist in areas such as:

- Route guards.
- Design system.
- Delivery estimate widget.
- Admin dashboard hardening.
- Vendor home command center.

Backend test scripts include:

- Server health/API tests.
- Push notification tests.
- Email checks.
- Flash sale checks.
- Performance test script.

16.2 Quality Checks Used for This Documentation

For this documentation package:

- Existing source files and route maps were inspected.
- Current feature docs were consolidated.
- Current backend route registration was reviewed.
- Frontend page/layout structure was reviewed.
- PDF generation was implemented with a reusable script.

16.3 Recommended Production Test Plan

Before production release, run:

1. Client build.
2. Client unit/component tests.
3. Server unit/API tests.
4. Authentication guard tests.
5. Checkout discount persistence test.

6. Order creation and vendor split test.
7. Payment verification workflow test.
8. Courier assignment workflow test.
9. Return/refund/vendor deduction test.
10. Admin RBAC and staff permission test.
11. Shop storefront/follow test.
12. Notification event test.
13. Analytics emission test.
14. Mobile responsive smoke test.

Chapter 17: Deployment and Environment Configuration

17.1 Client Deployment

The client is a Vite React app and can be deployed to providers such as Netlify, Vercel, or static hosting. It uses environment variables for API base URL and frontend configuration.

17.2 Server Deployment

The server is a Node.js Express application. Deployment requires:

- Node.js runtime.
- MongoDB connection.
- Firebase Admin configuration.
- CORS allowed origins.
- Upload/storage configuration.
- Optional Redis.
- Optional email/push credentials.
- Optional payment/courier credentials.

17.3 Environment Safety

Never publish real secrets in documentation, screenshots, or public repositories. Use `.env.example` only for variable names and placeholder values.

Important environment groups:

- MongoDB URL.
- Firebase service credentials.
- JWT/API secrets.
- CORS origins.
- Client/server URLs.
- Courier provider tokens.
- Payment provider keys.
- Email credentials.
- Push VAPID keys.
- Storage provider credentials.
- Redis URL.

17.4 Operational Dependencies

The marketplace can run with partial integrations, but production reliability improves when these are configured:

- Stable MongoDB.
- Redis for queues.
- Email provider.
- Push keys.
- Payment webhook verification.
- Courier provider credentials.
- Storage/CDN provider.
- Error logging and monitoring.

Chapter 18: Documentation and Training Layer

18.1 Existing Documentation Files

The repository includes documentation such as:

- `README.md`
- `README_ADMIN.md`
- `README_VENDOR.md`
- `README_USER.md`
- `ROLE_FEATURES_DOCUMENTATION.md`
- `MARKETPLACE_WORKFLOW_DIAGRAMS.md`
- `MARKETPLACE_ROLE_WORKFLOWS.md`
- `DARAZ_LEVEL_MARKETPLACE_AUDIT.md`
- `DARAZ_PROFESSIONAL_FEATURE_GAP_STATUS.md`
- `TESTING_DOCUMENTATION.md`
- `COURIER_INTEGRATION_WORKFLOW.md`
- Dynamic category/product docs.

18.2 Amiyo-Go University

Amiyo-Go University provides role learning:

- Public `/university`: customer-only learning content.
- Protected `/vendor/university`: vendor learning content.
- Protected `/admin/university`: admin/operator learning content.

This protects internal operating information from public users while still giving each role contextual guidance.

Chapter 19: Production-Readiness Assessment

19.1 Strong Areas

Current strong areas:

- Role-based marketplace architecture.
- Large admin control surface.
- Vendor seller-center pages.
- Customer shopping and post-purchase flows.
- Shop storefront feature.
- Discount persistence improvements.
- Logistics state machine foundations.
- COD and reverse logistics state foundations.
- Promotion, flash sale, loyalty, and growth foundations.
- Trust and safety foundations.
- Analytics/reporting foundations.
- Staff/RBAC patterns.
- Documentation and training layer.

19.2 Remaining Hardening Areas

High-value production hardening still recommended:

- Make BullMQ event bus universal for all order/payment/return/support/logistics events.
- Expand failed-job and failed-notification monitoring UI.
- Strengthen end-to-end tests for every admin queue.
- Add more complete payment provider webhook coverage.
- Add real courier API production booking and tracking after provider selection.
- Improve analytics emission audit for every state transition.
- Add more observability for queue lag, stale dashboards, and failed integrations.
- Add stricter secret scanning and environment validation.
- Add formal release checklist and rollback procedure.

19.3 Market Launch Readiness

Amiyo-Go has enough functional breadth to operate a controlled marketplace pilot if:

- MongoDB is stable.
- Auth and CORS are correctly configured.
- Manual payment verification is operational.
- Manual/internal courier workflow is accepted initially.
- Admin staff permissions are configured carefully.
- Vendors are onboarded with real KYC review.
- Support and returns policies are clearly published.

- Operational staff follow queue-based procedures.

  For high-scale public launch, the hardening areas in section 19.2 should be prioritized.

Chapter 20: Limitations and Future Work

20.1 Current Limitations

Known limitations include:

- Some integrations are adapter-ready but not fully live without credentials.
- Redis/BullMQ is not yet universal across every workflow.
- Some analytics dashboards depend on consistent event emission.
- Some advanced Daraz-level features are present as foundations but need deeper automation.
- External courier tracking depends on provider API support.
- Payment gateway behavior depends on provider setup and webhook reliability.
- Some queue and dashboard views require production data to validate deeply.

20.2 Future Work

Future work should include:

- Full event bus coverage.
- Production courier adapters.
- Payment gateway webhook hardening.
- Server-side cart persistence with guest merge.
- Advanced sponsored product advertising.
- Map-pin address improvements.
- Deep seller health score automation.
- More formal fraud thresholds.
- Scheduled reports and data quality alerts.
- A/B testing analytics.
- More comprehensive Playwright/Cypress end-to-end tests.
- Production monitoring dashboards.
- Automated backup and restore plan.

Chapter 21: Conclusion

Amiyo-Go is structured as a real marketplace operating system rather than a basic ecommerce app. It supports customer shopping, seller-center operations, and admin governance through role-specific pages, backend domain modules, data models, and workflows.

The current implementation includes strong foundations for catalog, search, checkout, orders, vendor operations, admin operations, logistics, COD, returns, payments, promotions, loyalty, notifications, trust, analytics, staff permissions, shop storefronts, and documentation. The system can operate without paid courier APIs by using manual/internal logistics workflows and can later attach provider APIs through adapter services.

The next stage should focus on reliability, observability, end-to-end testing, queue unification, payment/courier production hardening, and data quality. With those improvements, Amiyo-Go can move closer to a production-grade Daraz-level marketplace.

Appendices

Appendix A: Feature Matrix

```
| Feature Area | Customer | Vendor | Admin |
| Account/profile | Yes | Yes | Staff/admin |
| Saved addresses | Yes | Pickup/shop address | View/manage where needed |
| Product browse | Yes | Own product view | All products |
| Product creation | No | Yes | Admin edit/moderate |
| Product moderation | No | Submission/rejection view | Yes |
| Shop storefront | Browse/follow | Manage own shop | Control visibility/settings |
| Cart/checkout | Yes | No | Order oversight |
| Vouchers/promos | Apply | Create/join where enabled | Create/manage |
| Orders | Own orders | Vendor orders | Global orders |
| Packing/dispatch | Track only | Pack/pickup-ready | Monitor/override |
| Courier assignment | View | View/request where enabled | Assign/reassign |
| COD | Pay at delivery | Settlement visibility | Reconcile |
| Returns | Request | Respond/inspect | Decide/monitor/refund |
| Reviews/Q&A | Ask/review | Reply/manage | Moderate |
| Support | Create tickets | Vendor messages | Queue/assign/reply |
| Finance | Order payment view | Earnings/payouts | Payout/finance controls |
| Analytics | Personal surfaces | Vendor reports | Full platform reports |
| Trust/safety | Report issues | Respond to cases | Full queue/enforcement |
| University | Customer guide | Protected vendor guide | Protected admin guide |
```

Appendix B: Important Commands

```
Client:
cd Client
npm run dev
npm run build
npm run test
Server:
cd Server
npm run dev
npm start
npm test
npm run seed
npm run make:admin
Documentation PDF:
node scripts/generateThesisPdf.js
```

Appendix C: Key Route Examples

```
Customer:
/products
/product/:id
/shops
/shops/:slug
/cart
/checkout
/orders
/orders/:orderId
/returns
/support
/university
Vendor:
/vendor/dashboard
/vendor/shop/settings
```

```
/vendor/products
/vendor/orders
/vendor/finance
/vendor/marketing
/vendor/reports
/vendor/university
Admin:
/admin
/admin/operations
/admin/products
/admin/vendors
/admin/orders
/admin/returns
/admin/logistics
/admin/finance
/admin/payouts
/admin/trust-safety
/admin/analytics
/admin/staff
/admin/university
```

Appendix D: Glossary

```
| Term | Meaning |
| COD | Cash on delivery |
| RTO | Return to origin |
| KYC | Know your customer/business verification |
| GMV | Gross merchandise value |
| SLA | Service level agreement |
| RBAC | Role-based access control |
| Vendor order | Vendor-specific split of a customer order |
| Promotion snapshot | Saved discount rule result for historical order accuracy |
| Reverse logistics | Return shipment workflow after delivery |
| Queue-first operations | Admin pattern where staff handle exception queues before routine browsing |
```

Appendix E: Source Documentation References

This thesis documentation consolidates information from the project source code and existing docs:

- `README.md`
- `README_ADMIN.md`
- `README_VENDOR.md`
- `README_USER.md`
- `ROLE_FEATURES_DOCUMENTATION.md`
- `MARKETPLACE_WORKFLOW_DIAGRAMS.md`
- `MARKETPLACE_ROLE_WORKFLOWS.md`
- `DARAZ_LEVEL_MARKETPLACE_AUDIT.md`
- `DARAZ_PROFESSIONAL_FEATURE_GAP_STATUS.md`
- `TESTING_DOCUMENTATION.md`
- `COURIER_INTEGRATION_WORKFLOW.md`
- `Server/index.js`
- `Server/routes`
- `Server/models`
- `Server/services`
- `Client/src/pages`
- `Client/src/components`
- `Client/src/layouts`
- `Client/src/routes`